

Introduction:

This project is a Python-based Tic-Tac-Toe game developed in conjunction with the Pygame library. In this game, the user competes against a computer-controlled opponent, using an interactive graphical interface. This game is RPG-inspired, focusing on creating a visually engaging and themed experience in addition to implementing the core game logic. In particular, the theme is inspired by a garden environment, where the user helps a Mr. Mantis character protect his garden from scarabs with the help of ladybugs.

The project is divided into two respective regions: frontend and backend. The backend tracks game state, player interaction, score tracking, winner detection and computer decisions. The frontend displays artwork, processes user input, updates the screen, and renders graphical interface.

Features:

1) Player vs. Computer:

The user competes against a computer-controlled opponent. The game alternates sides, starting with the player's turn. The player places ladybugs on the board while the computer places scarab pieces.

2) Computer Opponent:

The computer opponent uses rule-based decision making, as opposed to the original random-selection design. The computer has three modes: block, winning, or random. Firstly, the computer will try to block the player from winning, then it will prioritize important board positions, such as empty spaces in rows it has scarabs placed on. Otherwise, it will randomly select a square on the board.

3) Score tracking

For each round, the game will keep a running score of player wins, computer wins, and game ties. Scores can be reset by clicking on the “THUNDERSTORM” button, which completely resets the game.

4) Automatic reset

After each round ends, the game will reset the board and preserve the current score. This feature is automatic and allows players to continue playing without resetting their manually.

5) Themed UI

This game is RPG-inspired and replaces traditional X and O symbols with ladybugs and scarabs. The theme for this UI is garden-themed; consequently, additional decorative elements such as a praying mantis character, as well as clouds, help create a cohesive visual experience.

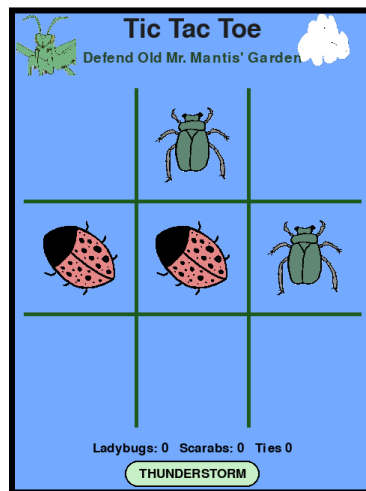


Figure 1: Themed UI

Backend Architecture:

For the backend, the application initializes and creates a blank Tic-Tac-Toe board. A 3x3 data structure, or board, stores the game state. Each cell within the 3x3 data structure holds "X", "O", or is empty " ". The player is represented by "X" and the computer opponent is represented by "O".

```
board = [  
    ["", "", ""],  
    ["", "", ""],  
    ["", "", ""]  
]  
  
# list of lists like a 2D array  
  
# initializing the game_state variables!  
game_over = False  
winner = None  
player = "X" # user is X  
computer = "O"  
CELL_SIZE = 150  
  
player_score = 0  
computer_score = 0  
ties = 0
```

Figure 2: blank 3x3 Tic Tac Toe board

As the user interacts with the UI by clicking through the cells to place X marks, the game system checks for wins, at which point the round ends. While the round remains active, the backend stores the variables `game_over`, `winner`, `player_score`, `computer_score`, and `ties`, which allow the program to track the status of the game, informing decisions about whether moves should be accepted or not, and whether the score should be updated.

Several functions exist in the backend. The `handle_click()` function takes in the user's mouse clicks and converts them into rows and columns on the tic tac toe board. If the selected

cell is empty and the game is active, the board accepts this input and a cell is marked with “X” or “O”. The backend then checks whether the player’s move caused a win or tie.

```
def handle_click(pos):
    global computer_pending, computer_move_time, reset_pending, reset_time
    global winner, game_over
    x, y = pos

    board_x = 25 # left edge of the board
    board_y = 105 # top of the board

    #is the click inside the board?
    if board_x <= x <= board_x + 450 and board_y <= y <= board_y + 450:

        col = (x - board_x) // CELL_SIZE
        row = (y - board_y) // CELL_SIZE #dont really understand this, the logic

        print("Clicked:", row, col)

        #if empty, place an X

        if board[row][col] == "" and not game_over and not computer_pending:
            board[row][col] = player

            result = check_winner()
            if result is not None:
                winner = result
                game_over = True
                update_score(result)

                reset_pending = True
                reset_time = pygame.time.get_ticks() + 2500
                print("Game Over: ", winner)
                print("Player Score: ", player_score)
                print("Computer Score: ", computer_score)
                print("Ties Score: ", ties)
                return

            computer_pending = True
            computer_move_time = pygame.time.get_ticks() + 400
            return
```

Figure 3: handle_click() function

The check_winner() function checks all rows, columns, and diagonals, checking to see if three matching symbols are present. If true, the function returns the winner, either “X” or “O”. If the board is full and no winner has been selected, the function returns “Tie.”

```

def check_winner():
    for row in board: #check row for row
        if row[0] == row[1] == row[2] != "":
            return row [0] #return either X or O

    for col in range(3): # check column for column
        if board[0][col] == board [1][col] == board[2][col] != "":
            return board[0][col] #value in column

    if board[0][0] == board[1][1] == board[2][2] != "":
        return board[0][0]

    if board[0][2] == board [1][1] == board[2][0] != "":
        return board[0][2]

    for row in board: #is the row full?
        if "" in row:
            return None

    return "Tie" #check for tie

```

Figure 4: check_winner() function

The computer_move() function makes decisions regarding the computer-opponent's behavior. It evaluates the board state by checking whether the player is about to fill a row and consequently blocking the player when necessary. It also checks whether the computer itself is about to fill a row and prioritizes winning and making useful moves, such as filling in the center squares. Otherwise, it chooses a random available position.

```

def computer_move():
    #computer tries to win, it will try to complete a row
    for row in range(3):
        for col in range(3):
            if board[row][col] == "":
                board[row][col] = computer
                if check_winner() == computer:
                    print ("Computer moved (win):", row, col)
                    return
                board[row][col] = ""
    #computer will try to block the player
    for row in range(3):
        for col in range(3):
            if board[row][col] == "":
                board [row][col] = player
                if check_winner() == player:
                    board[row][col] = computer
                    print("Computer moved (block):", row, col)
                    return
                board[row][col] = ""
    #computer will try to take center
    if board[1][1] == "":
        board[1][1] = computer
        print("Computer moved (center)")
        return

    #otherwise random
    empty_spaces = []

    for row in range(3):
        for col in range(3):
            if board[row][col] == "":
                empty_spaces.append((row, col))

    if empty_spaces:
        row, col = random.choice(empty_spaces)
        board[row][col] = computer
        print("Computer moved: ", row, col)

```

Figure 5: computer_move() function

The backend also includes `reset_game()` and `reset_all()`, which comprise the backend's reset logic. The function `reset_game()` clears the board and saves the game score in between rounds. The function `reset_all()` resets both the game score and simultaneously begins a new round.

Frontend Architecture:

The frontend of this project uses Pygame library to build the game display, everything the user sees and interacts with. It starts by setting the main Pygame window to a fixed size and color. For the purposes of this garden-themed game, the selected color was blue. The function `draw_grid()` was used to draw a 3x3 grid, with each cell being sized according to the board positions defined in the backend.

Custom hand-drawn images were used in place of X and O. In the game, the player's moves are represented by ladybugs while the computer's moves are represented by scarabs. The function `draw_marks()` was used to display both images. Simultaneously, `draw_title()` was used to display the game's title and praying mantis character. These images were resized and loaded onto the window, with the ladybugs and scarabs being centered inside each cell in the grid.

Two additional drawing functions were included: `draw_status()` and `draw_restart_button()`. The function `draw_status()` neatly displays game messages and score values, stored in the game's backend. The `draw_restart_button()` function was used to display a reset button, which calls the backend function `reset_all()` and simulates a "thunderstorm" to clear the board.

Challenges:

Challenges encountered during the development of this game was handling mouse input correctly and consistently. Pixel coordinates were used to map the specific board positions. Careful debugging ensured that clicks matched the boundaries of each cell.

Additionally, creation of the graphical interface introduced new challenges. Custom images were created for this game, and were quite demanding to introduce into the game. Specifically, they needed to be resized and placed correctly within the board cells and the window in general, so as to not interfere with other game elements. The placement of titles and status messages, and score displays required frequent adjustments to create a cohesive and visually pleasing display, which proved tedious at times.

Future Improvements

1) Multiple difficulty levels

Right now, the game only has one difficulty level. One possible improvement for future iterations of this game would be to create multiple levels of difficulty. The player could start by playing against a computer opponent that randomly selects places on the grid. As more rounds go on, the computer makes rule-based decisions, which is more difficult than randomly selecting different places on the grid.

2) Sound effects

Currently, the game does not feature any sound effects. However, future iterations could have sound effects for player wins and losses.

3) Mr. Mantis commentary

Mr. Mantis dialogue was a scrapped feature for this game. Depending on the current game state, Mr. Mantis would react to the player winning, losing, or being tied.

Additionally, random garden-messages and Mr. Mantis commentary was planned to make the game feel more alive and interactive.

4) Better custom images

Future versions of the game could use higher-quality graphics for the custom-designed images used to depict Mr. Mantis, ladybugs, and scarabs. A more consistent artistic style would further enhance game experience and immersion.